Unsupported Cell Type. Double-Click to inspect/edit the content.

```
### Data analysis
import numpy as np
import pandas as pd

#### Visualisation
import matplotlib.pyplot as plt

### StandardScalar
from sklearn.preprocessing import StandardScaler
### KNN
from sklearn.neighbors import KNeighborsClassifier
### Performance metric
from sklearn.metrics import confusion_matrix, accuracy_score, precision_sc
```

```
### Load the data
df_train = pd.read_csv("train_KNN.csv")
df_test = pd.read_csv("test_KNN.csv")
```

```
df_train.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 118.9 KB
```

```
df_test.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  418 non-null    int64
 1   Survived     418 non-null    int64
 2   Pclass       418 non-null    int64
 3   Name         418 non-null    str
 4   Sex          418 non-null    str
 5   Age          332 non-null    float64
 6   SibSp        418 non-null    int64
 7   Parch        418 non-null    int64
 8   Ticket       418 non-null    str
 9   Fare         417 non-null    float64
 10  Cabin        91 non-null     str
 11  Embarked     418 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 56.1 KB
```

```
df_train.isnull().sum()
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

```
mp = 100*(df_train.isnull().sum()/len(df_train))
mp
```

```
PassengerId     0.000000
Survived        0.000000
Pclass          0.000000
Name            0.000000
Sex             0.000000
Age            19.865320
SibSp           0.000000
Parch           0.000000
Ticket          0.000000
Fare            0.000000
Cabin          77.104377
Embarked        0.224467
dtype: float64
```

```
mp = mp[mp>0].sort_values()
```

```
mp
```

```
Embarked     0.224467
Age         19.865320
Cabin       77.104377
dtype: float64
```

```
df_train["Age"].median()
```

```
28.0
```

```
df_train["Fare"].median()
```

```
14.4542
```

```
df_train[df_train["Embarked"].isnull()]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
61	62	1	1	Icard, Miss. Amelie	female	38.0	0	0	113572	80.0	B28	NaN
829	830	1	1	Stone, Mrs. George Nelson (Martha Evelyn)	female	62.0	0	0	113572	80.0	B28	NaN

```
df_train = df_train.dropna(axis = 0, subset= ["Embarked"])
```

```
df_train.info()
```

```
<class 'pandas.DataFrame'>
Index: 889 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  889 non-null    int64
 1   Survived     889 non-null    int64
 2   Pclass       889 non-null    int64
 3   Name         889 non-null    str
 4   Sex          889 non-null    str
 5   Age          712 non-null    float64
 6   SibSp        889 non-null    int64
 7   Parch        889 non-null    int64
 8   Ticket       889 non-null    str
 9   Fare         889 non-null    float64
 10  Cabin        202 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 125.8 KB
```

```
df_train["Age"] = df_train["Age"].fillna(df_train["Age"].median())
```

```
df_train = df_train.drop(["Cabin"], axis = 1)  ## 1 is for column
```

```
####Testing
```

```
mpt = 100*(df_test.isnull().sum()/len(df_test))
mpt
```

```
PassengerId     0.000000
Survived        0.000000
Pclass          0.000000
Name            0.000000
Sex             0.000000
Age            20.574163
SibSp           0.000000
Parch           0.000000
Ticket          0.000000
Fare            0.239234
Cabin          78.229665
Embarked        0.000000
dtype: float64
```

```
mpt = mpt[mpt>0].sort_values()
mpt
```

```
Fare      0.239234
Age      20.574163
```

```
Cabin    78.229665
dtype: float64
```

```
df_test[df_test["Fare"].isnull()]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
152	1044	0	3	Storey, Mr. Thomas	male	60.5	0	0	3701	NaN	NaN	S

```
df_test = df_test.dropna(axis = 0, subset= ["Fare"])
```

```
df_test.info()
```

```
<class 'pandas.DataFrame'>
Index: 417 entries, 0 to 417
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  417 non-null    int64
 1   Survived     417 non-null    int64
 2   Pclass       417 non-null    int64
 3   Name         417 non-null    str
 4   Sex          417 non-null    str
 5   Age          331 non-null    float64
 6   SibSp        417 non-null    int64
 7   Parch        417 non-null    int64
 8   Ticket       417 non-null    str
 9   Fare         417 non-null    float64
 10  Cabin        91 non-null     str
 11  Embarked     417 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 59.3 KB
```

```
df_test["Age"] = df_test["Age"].fillna(df_test["Age"].median())
```

```
df_test = df_test.drop(["Cabin"], axis = 1)  ## 1 is for column
```

```
df_train.drop(["PassengerId","Name","Ticket"], axis = 1, inplace = True)
```

```
df_test.drop(["PassengerId","Name","Ticket"], axis = 1, inplace = True)
```

```
df_train.info()
```

```
<class 'pandas.DataFrame'>
Index: 889 entries, 0 to 890
Data columns (total 8 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Survived  889 non-null    int64
 1   Pclass    889 non-null    int64
 2   Sex       889 non-null    str
 3   Age       889 non-null    float64
 4   SibSp     889 non-null    int64
 5   Parch     889 non-null    int64
 6   Fare      889 non-null    float64
 7   Embarked  889 non-null    str
dtypes: float64(2), int64(4), str(2)
memory usage: 67.7 KB
```

```
df_test.info()
```

```
<class 'pandas.DataFrame'>
Index: 417 entries, 0 to 417
Data columns (total 8 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Survived  417 non-null    int64
 1   Pclass    417 non-null    int64
 2   Sex       417 non-null    str
 3   Age       417 non-null    float64
 4   SibSp     417 non-null    int64
 5   Parch     417 non-null    int64
 6   Fare      417 non-null    float64
 7   Embarked  417 non-null    str
dtypes: float64(2), int64(4), str(2)
memory usage: 31.8 KB
```

```
df_train["Survived"] = df_train["Survived"].apply(str)
```

```
df_train["Pclass"] = df_train["Pclass"].apply(str)
```

```
df_test["Survived"] = df_test["Survived"].apply(str)
df_test["Pclass"] = df_test["Pclass"].apply(str)
```

```
df_train_num = df_train.select_dtypes(exclude = "object")
df_train_obj = df_train.select_dtypes(include = "object")
```

```
C:\Users\DurgaSharma\AppData\Local\Temp\ipykernel_11740\4062479843.py:2: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specifie
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
```

```
  df_train_obj = df_train.select_dtypes(include = "object")
```

```
df_test_num = df_test.select_dtypes(exclude = "object")
df_test_obj = df_test.select_dtypes(include = "object")
```

```
C:\Users\DurgaSharma\AppData\Local\Temp\ipykernel_11740\1504789278.py:2: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specifie
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  df_test_obj = df_test.select_dtypes(include = "object")
```

```
df_train_num.columns
```

```
Index(['Age', 'SibSp', 'Parch', 'Fare'], dtype='str')
```

```
#####Dummification of the data

df_train_obj = pd.get_dummies(df_train_obj, drop_first = True)
```

```
#####Dummification of the data

df_test_obj = pd.get_dummies(df_test_obj, drop_first = True)
```

```
df_test_obj.columns
```

```
Index(['Survived_1', 'Pclass_2', 'Pclass_3', 'Sex_male', 'Embarked_Q',
       'Embarked_S'],
      dtype='str')
```

```
#### Final processed data set

Final_train_df = pd.concat([df_train_num,df_train_obj, df_train["Survived"
Final_train_df.head()
```

	Age	SibSp	Parch	Fare	Survived_1	Pclass_2	Pclass_3	Sex_male	Embarked_Q	Embarked_S	Survived
0	22.0	1	0	7.2500	False	False	True	True	False	True	0
1	38.0	1	0	71.2833	True	False	False	False	False	False	1
2	26.0	0	0	7.9250	True	False	True	False	False	True	1
3	35.0	1	0	53.1000	True	False	False	False	False	True	1
4	35.0	0	0	8.0500	False	False	True	True	False	True	0

```
#### Final processed data set

Final_test_df = pd.concat([df_test_num,df_test_obj, df_test["Survived"]],
Final_test_df.head()
```

	Age	SibSp	Parch	Fare	Survived_1	Pclass_2	Pclass_3	Sex_male	Embarked_Q	Embarked_S	Survived
0	34.5	0	0	7.8292	False	False	True	True	True	False	0
1	47.0	1	0	7.0000	True	False	True	False	False	True	1
2	62.0	0	0	9.6875	False	True	False	True	True	False	0
3	27.0	0	0	8.6625	False	False	True	True	False	True	0
4	22.0	1	1	12.2875	True	False	True	False	False	True	1

```
#### Standarised the data
### define Xtrain, Y train

X_train = Final_train_df.drop("Survived", axis =1)
```

```
y_train = Final_train_df["Survived"]
```

```
y_tes = Final_test_df["Survived"]
```

```
#### Standarised the data
### define Xtrain, Y train

X_test = Final_test_df.drop("Survived", axis =1)
```

```
scaler = StandardScaler()
scaler.fit(X_train)
```

```
StandardScaler
▸ Parameters
```

```
scaled_X_train = scaler.transform(X_train)
```

```
scaled_X_test = scaler.transform(X_test)
```

```
##### Use the model
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(scaled_X_train, y_train)
```

```
KNeighborsClassifier
▸ Parameters
```

```
y_pred = knn.predict(scaled_X_test)
```

```
accuracy_score(y_tes, y_pred)
```

```
0.9928057553956835
```

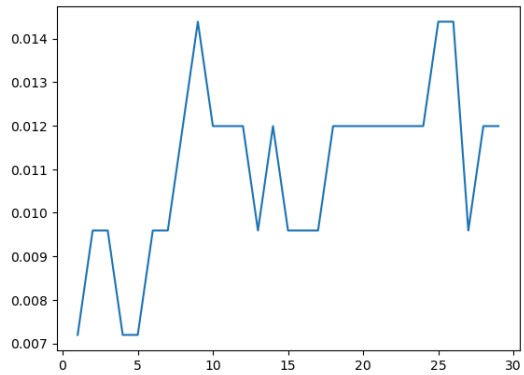
```
#### for different values of k we are going to check
Performance_Metric = []

for k in range(1,30):
    knn_k = KNeighborsClassifier(n_neighbors=k)
    knn_k.fit(scaled_X_train, y_train)
    y_pred_k = knn_k.predict(scaled_X_test)
    accuracy_k = 1-accuracy_score(y_tes, y_pred_k)
    Performance_Metric.append(accuracy_k)
```

```
Performance_Metric
```

```
[0.007194244604316502,
 0.00959232613908878,
 0.00959232613908878,
 0.007194244604316502,
 0.007194244604316502,
 0.00959232613908878,
 0.00959232613908878,
 0.011990407673860948,
 0.014388489208633115,
 0.011990407673860948,
 0.011990407673860948,
 0.011990407673860948,
 0.00959232613908878,
 0.011990407673860948,
 0.00959232613908878,
 0.00959232613908878,
 0.00959232613908878,
 0.011990407673860948,
 0.011990407673860948,
 0.011990407673860948,
 0.011990407673860948,
 0.011990407673860948,
 0.011990407673860948,
 0.011990407673860948,
 0.014388489208633115,
 0.014388489208633115,
 0.00959232613908878,
 0.011990407673860948,
 0.011990407673860948]
```

```
plt.plot(range(1,30), Performance_Metric)
```

```
[<matplotlib.lines.Line2D at 0x2595c1db1d0>]
```




```
lr, dt
```